Table des matières

Labo 02 — Concevoir l'étoile, puis la prototyper en SQL 1
Avant de commencer 2
Étape 1 — Déclarer le grain des six processus 2
Étape 2 — `dim_date`, et le piège des libellés 3
Étape 3 — `dim_concessionnaire` 4
Étape 4 — `dim_produit` 6
Étape 5 — `fct_ventes`, au grain de la ligne 7
Étape 6 — La question du métier 9
Étape 7 — La matrice de bus 10
Récapitulatif 10
Dépannage 11

Labo 02 — Concevoir l'étoile, puis la prototyper en SQL

Labo · Module 2/9 — Formation Analytics Engineering

Ce que vous allez faire. Déclarer le grain des six processus de Boréal, puis prototyper la première étoile en SQL pur, sans aucun outil : une table de dates avec ses libellés en français, deux dimensions, et un fait au grain de la ligne de commande. Vous finirez par la question du métier, celle qui demandait cinq jointures sur le brut, et qui tiendra ici en une requête. Objectif de fin : 75 652 lignes de fait pour 430 148 012,16 $, et une matrice de bus qui dit ce que les trois étoiles partagent.

Un seul terminal sert dans ce labo : la ligne de commande DuckDB, ouverte sur l'entrepôt depuis `Projet_Boreal`. Aucun outil ne s'installe ici, et c'est voulu : la conception se fait avant dbt, pas avec lui.

La liste de contrôle du cours :

- ☐ Le grain des six processus déclaré, une phrase chacun
- ☐ Schéma `prototype` créé dans l'entrepôt
- ☐ `dim_date` : 1 551 jours, du 2022-01-01 au 2026-03-31, libellés en français
- ☐ `dim_concessionnaire` : 156 lignes, 156 clés, 0 palier non décodé

© 2026 iAlexMG. Tous droits réservés.

- ☐ `dim_produit` : 68 articles, quatre gammes
- ☐ `fct_ventes` : 75 652 lignes, grain vérifié, 430 148 012,16 $
- ☐ La question du métier répondue en une requête
- ☐ La matrice de bus remplie pour les six processus

Le labo se fait en deux séances. La première va jusqu'à l'étape 6, la seconde porte la matrice de bus.

Avant de commencer

Ce labo suppose la couche `RAW` montée au Cours 01. Une vérification, dans la ligne de commande DuckDB ouverte depuis `Projet_Boreal` :

```
duckdb plateforme/entrepot/boreal.duckdb
```

```
select count(*) as tables_raw from duckdb_tables() where schema_name = 'raw';
```

Attendu : 22. Si le compte diffère, reprenez l'étape 1 du Labo 01.

Vous allez écrire dans un schéma à part, `prototype`, et non dans `marts`. La raison tient à ce qu'est ce labo : un brouillon de conception, jeté une fois que dbt bâtira les vraies tables au Cours 04. Le Cours 03 vous dira quand l'abandonner.

```
create schema if not exists prototype;
```

Étape 1 — Déclarer le grain des six processus

Théorie : `Theorie_02_AE.md`, sections 2, 3 et 11.

Cette étape ne s'écrit pas en SQL. Prenez une feuille, ou un fichier texte, et écrivez six phrases, une par processus. Chacune doit pouvoir se lire à voix haute et commencer par « une ligne représente ».

Les six processus de Boréal :

Le processus	**Où la donnée vit**
Un concessionnaire commande	`raw.erp_vbak` et `raw.erp_vbap`
Un véhicule est assemblé et expédié	`raw.erp_zserie`

© 2026 iAlexMG. Tous droits réservés.

Le processus	Où la donnée vit
Un véhicule est vendu au détail	`raw.garanties_enregistrements`
Une réclamation suit son cycle	`raw.garanties_reclamations`
Un véhicule connecté publie une sortie	`raw.telemetrie_sorties`
Un actif de l'usine passe une heure	schéma `argent`, livré au Cours 08 de Data Engineering

Écrivez vos six phrases avant de lire la suite. Comparez ensuite avec la section 11 de la théorie : si l'une des vôtres diffère, demandez-vous laquelle des deux permet la question la plus fine. Ce n'est pas un exercice de devinette, c'est le geste que vous ferez devant chaque projet neuf.

Une vérification vaut la peine d'être faite en SQL, celle du grain le plus fin disponible pour les ventes :

```
select
    count(*) as lignes,
    count(distinct vbeln) as commandes,
    count(distinct (vbeln, posnr)) as couples_commande_poste
from raw.erp_vbap;
```

Trois nombres sortent : 81 292, 25 483 et 77 732. Le premier est le nombre de lignes de la `RAW`, livraisons multiples comprises, et il ne correspond à aucun grain : c'est un compte de livraison, pas un compte d'objets. Les deux autres sont les deux grains possibles, le grain « commande » à 25 483 et le grain « ligne de commande » à 77 732. Vous prendrez le second.

Étape 2 — `dim_date`, et le piège des libellés

Théorie : `Theorie_02_AE.md`, section 9.

Toute étoile commence par une dimension de dates, parce que tous les faits la partagent. Elle ne se tire d'aucune source : elle se fabrique.

Commencez par constater le piège, avant de l'éviter :

```
select
    strftime(date '2022-01-01', '%B') as mois_strftime,
    monthname(date '2022-01-01') as mois_monthname,
    dayname(date '2022-01-01') as jour_dayname;
```

Trois fonctions, trois réponses en anglais : `January`, `January`, `Saturday`. DuckDB n'a pas de notion de langue, et aucune option ne change cela. Un entrepôt francophone qui s'appuie sur ces fonctions livre des mois en anglais à tous ses rapports, et le défaut se corrige ensuite dans chaque rapport, un par un.

© 2026 iAlexMG. Tous droits réservés.

La parade est de décoder une fois, à l'endroit unique qui fabrique la dimension :

```
create or replace table prototype.dim_date as
select
    date_jour,
    year(date_jour) as annee,
    quarter(date_jour) as trimestre,
    month(date_jour) as mois,
    ['janvier','février','mars','avril','mai','juin','juillet','août',
     'septembre','octobre','novembre','décembre'][month(date_jour)] as nom_mois,
    ['lundi','mardi','mercredi','jeudi','vendredi','samedi','dimanche'][isodow(date_jour)] as nom_jour,
    isodow(date_jour) >= 6 as est_fin_de_semaine
from (
    select unnest(generate_series(date '2022-01-01', date '2026-03-31', interval 1 day))::date as date_jour
);
```

Deux points méritent un mot. Les listes de DuckDB s'indexent à partir de 1, ce qui tombe bien : `month()` rend 1 pour janvier. Et `isodow()` rend 1 pour lundi et 7 pour dimanche, dans l'ordre de la norme ISO, alors que `dayofweek()` commence au dimanche : mélanger les deux décale toute la semaine.

Renvoi : créer une table depuis une requête est au Cours 03 de SQL, section 2 ; générer une série de dates, à son Cours 05, section 6.

Vérifiez :

```
select count(*) as jours, min(date_jour) as premier, max(date_jour) as dernier
from prototype.dim_date;

select annee, count(*) as jours from prototype.dim_date group by 1 order by 1;
```

Attendu	Valeur
Jours	1 551, du 2022-01-01 au 2026-03-31
2022, 2023, 2025	365 jours
2024	366 jours, c'est une année bissextile
2026	90 jours, la période s'arrête au 31 mars

Étape 3 — `dim_concessionnaire`

Théorie : `Theorie_02_AE.md`, sections 6 et 7.

© 2026 iAlexMG. Tous droits réservés.

Première dimension tirée d'une source. Trois gestes s'y superposent, et c'est normal : la conception les distingue, le SQL les fait ensemble.

```sql
create or replace table prototype.dim_concessionnaire as
with courant as (
    select *, row_number() over (partition by kunnr order by _lot desc) as rang
    from raw.erp_kna1
)
select
    md5(kunnr) as cle_concessionnaire,
    kunnr as code_concessionnaire,
    name1 as nom,
    ort01 as ville,
    regio as province,
    case kukla when '01' then 'Or' when '02' then 'Argent' when '03' then 'Bronze' end as palier,
    try_strptime(erdat, '%Y%m%d')::date as date_creation
from courant
where rang = 1;
```

Ce que fait chaque morceau :

Le morceau	**Ce qu'il fait**
`row_number() ... order by _lot desc`, puis `rang = 1`	tire l'état courant : la version du lot le plus récent
`md5(kunnr)`	fabrique la clé de substitution, ici par hachage de la clé naturelle
`case kukla ...`	décode le palier, d'après le dictionnaire de l'univers
`try_strptime(...)`	type la date SAP, avec son format

Vérifiez, et surtout vérifiez le décodage :

```sql
select
    count(*) as lignes,
    count(distinct cle_concessionnaire) as cles,
    count(*) filter (where palier is null) as paliers_non_decodes
from prototype.dim_concessionnaire;
```

Attendu : 156 lignes, 156 clés, 0 palier non décodé. La troisième colonne est celle qui compte : un `case` dont aucune branche ne correspond rend `NULL` sans rien signaler, et une dimension entière peut sortir vide de sens sans qu'aucune erreur n'apparaisse. Prenez l'habitude de compter les `NULL` après chaque décodage.

Renvoi : les fonctions de fenêtre, dont `row_number()`, sont au Cours 04 de SQL.

© 2026 iAlexMG. Tous droits réservés.

Piège — cette dimension est de type 1, et elle perd l'histoire. Telle qu'elle est écrite, elle ne garde que l'état courant : les 49 concessionnaires dont le palier a changé le 1er juillet 2024 apparaissent tous avec leur palier d'aujourd'hui, et les ventes de 2023 leur sont attribuées avec ce palier-là. C'est acceptable pour un prototype, et ce ne le sera plus au Cours 05, qui la passe en type 2.

Étape 4 — `dim_produit`

Théorie : `Theorie_02_AE.md`, section 5.

Même geste, avec deux décodages :

```sql
create or replace table prototype.dim_produit as
with courant as (
    select *, row_number() over (partition by matnr order by _lot desc) as rang
    from raw.erp_mara
)
select
    md5(matnr) as cle_produit,
    matnr as code_article,
    case matkl when 'MN' then 'Motoneige' when 'MM' then 'Motomarine'
               when 'HR' then 'Hors-route' when 'PAC' then 'Pièces et accessoires' end as gamme,
    case mtart when 'FERT' then 'Véhicule' when 'HAWA' then 'Pièce' end as type_article,
    nullif(zzannee_mod, '') as annee_modele
from courant
where rang = 1;

select gamme, type_article, count(*) as articles
from prototype.dim_produit
group by 1, 2
order by 1;
```

Gamme	Type	Articles
Hors-route	Véhicule	15
Motomarine	Véhicule	15
Motoneige	Véhicule	15
Pièces et accessoires	Pièce	23

© 2026 iAlexMG. Tous droits réservés.

Soixante-huit articles en tout. Remarquez ce que cette table n'est pas : il n'y a pas de table `dim_gamme` à côté, avec quatre lignes, vers laquelle `dim_produit` pointerait. La gamme est une colonne. C'est l'étoile plutôt que le flocon, et sur 68 lignes la question ne mérite pas d'être posée deux fois.

Étape 5 — `fct_ventes`, au grain de la ligne

Théorie : `Theorie_02_AE.md`, sections 3 et 4.

Le fait, enfin. Il joint les postes à leurs en-têtes, puis aux trois dimensions, et il écarte les postes refusés.

```
create or replace table prototype.fct_ventes as
with postes as (
    select *, row_number() over (partition by vbeln, posnr order by _lot desc) as rang
    from raw.erp_vbap
),
entetes as (
    select *, row_number() over (partition by vbeln order by _lot desc) as rang
    from raw.erp_vbak
)
select
    d.date_jour as cle_date,
    c.cle_concessionnaire,
    pr.cle_produit,
    p.vbeln as numero_commande,
    p.posnr as numero_poste,
    p.kwmeng::decimal(18, 3) as quantite,
    p.netwr::decimal(18, 2) as valeur_nette,
    p.wavwr::decimal(18, 2) as cout
from postes p
join entetes e on e.vbeln = p.vbeln and e.rang = 1
join prototype.dim_date d on d.date_jour = try_strptime(e.erdat, '%Y%m%d')::date
join prototype.dim_concessionnaire c on c.code_concessionnaire = e.kunnr
join prototype.dim_produit pr on pr.code_article = p.matnr
where p.rang = 1
  and (p.abgru is null or p.abgru = '');
```

Le premier contrôle est celui du grain. Il doit être fait à chaque fait que vous écrirez, et il tient en une requête : le nombre de lignes doit égaler le nombre de couples distincts qui définissent le grain.

```
select
    count(*) as lignes,
```


© 2026 iAlexMG. Tous droits réservés.

```
    count(distinct (numero_commande, numero_poste)) as couples_distincts
from prototype.fct_ventes;
```

Attendu : 75 652 et 75 652. Si les deux nombres diffèrent, une jointure a dupliqué des lignes, et votre fait ne dit plus ce que son grain annonce.

Puis les totaux :

```
select
    count(*) as lignes,
    count(distinct numero_commande) as commandes,
    sum(valeur_nette) as valeur_nette,
    sum(cout) as cout
from prototype.fct_ventes;
```

Mesure	Attendu
Lignes	75 652
Commandes	25 256
Valeur nette	430 148 012,16 $
Coût	312 784 347,46 $

Deux chiffres demandent une explication, et c'est là que le labo devient intéressant.

Les 75 652 lignes ne sont pas les 77 732 postes de la source : 2 080 postes sont refusés, par annulation du concessionnaire, par refus d'un délai ou par rupture chez le fournisseur, et le fait des ventes ne les compte pas.

Les 25 256 commandes ne sont pas les 25 483 de l'ERP non plus : 227 commandes n'ont que des postes refusés, et elles disparaissent entièrement du fait. C'est voulu, et c'est le genre d'écart qu'il faut savoir expliquer avant que quelqu'un ne le découvre dans un rapport.

```
with postes as (
    select *, row_number() over (partition by vbeln, posnr order by _lot desc) as rang
    from raw.erp_vbap
)
select
    count(*) as postes,
    count(*) filter (where abgru is not null and abgru <> '') as refuses,
    count(distinct vbeln) as commandes,
    count(distinct vbeln) filter (where abgru is null or abgru = '') as commandes_avec_un_poste_actif
from postes
where rang = 1;
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : 77 732 postes dont 2 080 refusés, 25 483 commandes dont 25 256 gardent au moins un poste actif.

Étape 6 — La question du métier

Théorie : `Theorie_02_AE.md`, section 1.

C'est le moment de vérifier que tout cela servait à quelque chose. Voici la question de la mise en situation : quelle gamme se vend le mieux au Québec, par trimestre, chez les concessionnaires de palier Or ?

```sql
select
    p.gamme,
    d.trimestre,
    sum(f.valeur_nette) as valeur_nette
from prototype.fct_ventes f
join prototype.dim_produit p on p.cle_produit = f.cle_produit
join prototype.dim_date d on d.date_jour = f.cle_date
join prototype.dim_concessionnaire c on c.cle_concessionnaire = f.cle_concessionnaire
where c.province = 'QC'
  and d.annee = 2025
  and c.palier = 'Or'
group by 1, 2
order by 1, 2;
```

Seize lignes, quatre gammes sur quatre trimestres. Les motoneiges :

Trimestre	Valeur nette
1	574 546,57 $
2	28 525,10 $
3	3 273 517,55 $
4	910 380,72 $

La saisonnalité saute aux yeux : les concessionnaires commandent leurs motoneiges l'été, pour les avoir en magasin l'hiver. Les motomarines font l'inverse, avec 2 224 614,99 $ au premier trimestre contre 41 641,32 $ au quatrième.

Comparez cette requête à ce qu'il aurait fallu écrire sur la `RAW` : cinq tables à joindre, trois codes à connaître, le lot en double à écarter, les montants à convertir. Ici, trois jointures sur des clés, et des libellés lisibles dans le `where`. C'est cela, la valeur du modèle dimensionnel, et c'est tout ce que le cours avait à démontrer.

© 2026 iAlexMG. Tous droits réservés.

Étape 7 — La matrice de bus

Théorie : `Theorie_02_AE.md`, sections 9, 10 et 11.

Dernière étape, et elle ne s'écrit pas non plus en SQL. Reprenez vos six phrases de l'étape 1, et dressez la matrice : une ligne par processus, une colonne par dimension, une croix quand le processus se découpe selon cette dimension.

Les huit dimensions de Boréal : `dim_date`, `dim_concessionnaire`, `dim_produit`, `dim_vehicule`, `dim_client`, `dim_actif`, `dim_anomalie`, `dim_politique_garantie`.

Remplissez-la avant de lire la section 7 du `PLAN_MODULES.md` de la formation, qui donne la réponse. Trois questions vous aideront à décider, pour chaque case : ce processus produit-il une donnée datée ? concerne-t-il un véhicule identifié par son numéro de série ? une personne est-elle en cause ?

Regardez ensuite les colonnes, et non les lignes. Celles qui portent le plus de croix sont les dimensions à construire en premier, puisque tout en dépend. Chez Boréal, `dim_date` les porte toutes, `dim_produit` cinq sur six, `dim_vehicule` quatre.

Regardez enfin les trous. Une case vide n'est pas une erreur : elle dit une question qu'on ne peut pas poser. La disponibilité d'un actif ne se découpe pas par concessionnaire, et c'est normal, un compresseur d'usine n'appartient à aucun concessionnaire.

Récapitulatif

L'état du prototype à la fin du labo :

Table	Lignes	Ce qu'elle porte
`prototype.dim_date`	1 551	du 2022-01-01 au 2026-03-31, libellés en français
`prototype.dim_concessionnaire`	156	l'état courant, type 1, palier décodé
`prototype.dim_produit`	68	quatre gammes, deux types
`prototype.fct_ventes`	75 652	le grain de la ligne, 430 148 012,16 $

Ce que vous savez faire maintenant, et qui ne dépend d'aucun outil : déclarer un grain, le vérifier, fabriquer une dimension avec sa clé de substitution, et reconnaître ce qu'un décodage raté laisse passer.

Le Cours 03 ouvre dbt et refait ce travail, mais autrement : versionné, testé, rejouable. Ce prototype aura servi à savoir où l'on va.

© 2026 iAlexMG. Tous droits réservés.

Dépannage

Le message ou le symptôme	Ce qu'il veut dire	La parade
`Binder Error: Referenced column "zzannee_mod" not found`	la colonne n'existe pas sous ce nom dans cette source	vérifier le nom exact dans le dictionnaire de l'univers
Le fait porte plus de lignes que de couples distincts	une jointure a dupliqué des lignes	vérifier que chaque jointure vers une dimension tombe sur une seule ligne
`paliers_non_decodes` n'est pas à 0	les codes du `case` ne correspondent pas à ceux de la source	relire le dictionnaire : les codes sont `01`, `02`, `03`
La valeur nette dépasse l'attendu de beaucoup	les postes refusés n'ont pas été écartés	ajouter la condition sur `abgru`
`Catalog Error: Table with name dim_date does not exist`	le schéma `prototype` n'a pas été créé, ou l'étape 2 a échoué	`create schema if not exists prototype;`, puis reprendre l'étape 2

© 2026 iAlexMG. Tous droits réservés.